Retry Pattern

Category: Resilience **One-liner:** Automatically re-attempt operations that failed for temporary reasons, spacing attempts out with backoff and randomness, so brief glitches don't become user-visible errors.

Part 0 — Concepts you need first

What is a "transient" failure?

Transient means temporary — it goes away on its own without anyone fixing anything. The opposite is **permanent**: it will fail identically no matter how many times you try.

In the table below, a **packet** is one small chunk of data sent over a network, the **request body** is the data sent with a request (e.g. a JSON order), a **load balancer** spreads requests across several **instances** (running copies) of a service, and **CPU** is the processor.

The distinction is the single most important idea in this chapter, because **retrying a transient failure is useful and retrying a permanent one is pure waste**.

Transient (retry helps)	Permanent (retry is useless)
Network dropped a packet	The password is wrong
Server briefly out of connections	The request body is malformed
Momentary CPU spike	You lack permission
Load balancer restarted an instance	The record doesn't exist

What is "backoff"?

Waiting between retry attempts instead of retrying instantly. **Exponential backoff** means each wait is longer than the last — typically doubling: 1s, 2s, 4s, 8s.

Why wait at all? Because retrying instantly against a struggling service adds **load** (the amount of work hitting it) at the worst possible moment, and gives it no time to recover.

What is "jitter"?

Deliberate randomness added to the wait time. Instead of every client waiting exactly 4 seconds, each waits a random duration around 4 seconds.

It sounds pointless until you see the problem it solves — see the thundering herd below.

What is the "thundering herd" problem?

Imagine a service has a two-second blip. During that blip, **ten thousand clients** fail. Every one of them starts a 4-second backoff at almost the same instant.

Four seconds later, **all ten thousand retry simultaneously**. The service, which had just recovered, is hit by a synchronised wall of traffic and falls over again. Now they all back off 8 seconds, and the same thing happens again, harder.

The clients have accidentally organised themselves into a **denial-of-service attack** (flooding a service with so many requests that it can't serve anyone). **Jitter breaks the synchronisation** by spreading those retries across a window.

What does "idempotent" mean?

An operation is **idempotent** if performing it multiple times has the same effect as performing it once.

- `set balance to 100` — **idempotent**. Do it five times, balance is 100.
- `add 100 to balance` — **not idempotent**. Do it five times, you've added 500.

This matters enormously for retries, because a retry might be the *second* execution of something that already succeeded. (Idempotency is also covered in System Design Fundamentals.)

Part 1 — The problem

In a **distributed microservices architecture** (an application split into many small services running on different machines and talking over a network), **failures are inevitable**. Services constantly communicate over networks and depend on numerous components, any of which can malfunction. Failures stem from unreliable networks, transient server **outages** (periods when a server is unavailable), overloaded load balancers, software bugs, even operator mistakes.

No matter how well you design, **it's virtually impossible to build a system that never fails**. Instead you design for **resilience** (the ability to keep working when parts fail), ensuring a small hiccup doesn't snowball into a major outage.

Transient failures specifically

These are short-lived errors that resolve on their own given a bit of time. A brief network glitch, a momentary loss of connectivity, a temporary service unavailability. Often these issues are **ephemeral** — try again after a short delay and it succeeds.

The classic example: a database connection failure caused by a spike in **concurrent** users (users active at the same moment). At the peak moment the connection is refused. **A few seconds later, after some connections close, a retry goes straight through.** Nothing was fixed; the condition passed.

Other common cases: network **timeouts** (giving up after waiting a set time), transient **HTTP 5xx** errors (status codes 500–599, meaning "the server failed"; 4xx codes mean "the request was wrong"), temporary resource exhaustion.

Why this needs solving

Without a strategy for such hiccups, microservices **propagate errors to end-users or upstream systems unnecessarily** (an **upstream** system is one that called you and is waiting on your answer). A tiny blip in connectivity could trigger a user-facing error — or require manual intervention — even though **the system might have recovered milliseconds later**.

That's the frustration the pattern removes: the failure was real, but it was over before the user finished reading the error message.

Part 2 — The solution

If an operation fails due to a temporary issue, **wait a short period and try again**, on the assumption the issue may have resolved in the interim.

Many distributed systems fail in **partial** ways — a subset of requests fail while others succeed — or suffer short-lived outages. Often, trying the same request again simply causes it to succeed.

A good illustration: if a service didn't respond because of a momentary spike, a retry a few moments later **might hit a less-busy instance** and succeed. With multiple instances behind a load balancer,

the retry isn't even necessarily going to the same place that failed.

The effect: retries **mask sporadic failures**, increasing the reliability the user perceives.

Why it works

Retries assume the failure was transient or **non-deterministic** (not guaranteed to happen the same way every time). Network blips clear up, **threads** (independent lines of execution in a program) get freed, services restart — so a subsequent attempt has a good chance of succeeding where the first failed. This dramatically reduces the error rate seen by higher-level services and end users. Instead of giving up immediately when a **downstream** call (a call to a service you depend on) fails, a service can retry and often complete the operation **without any manual fix**.

Part 3 — Transient vs. permanent: the critical distinction

Retry should only engage for errors **likely to be temporary**.

Worth retrying: network timeouts · HTTP 5xx server errors · 502 Bad Gateway from a load balancer · database **deadlocks** (two transactions each waiting for a lock the other holds; the database kills one) · connection refused (nothing is accepting connections right now) · **throttling** / **rate-limit** errors (the service is deliberately refusing because you're sending too much)

Not worth retrying: HTTP 400 (input validation failure) · HTTP 401 (**authentication** error — the caller couldn't prove who they are) · **business logic** errors (the request breaks a domain rule, e.g. insufficient funds) · configuration errors · fatal **exceptions** (errors thrown by code that can't be recovered from)

> **No amount of retrying will fix a bad request or invalid credentials.** Those errors are permanent *for that request*. Retrying them just repeats the failure and potentially worsens system load.

Therefore a well-designed retry mechanism **checks the error type or exception type**, retrying only transient conditions while letting permanent errors **fail fast** (return an error immediately rather than waiting).

Retry vs. fail-fast

An alternative strategy is **fail-fast**, typically managed via a Circuit Breaker (a component that stops calling a service after repeated failures, for a cooling-off period).

	Retry	Fail-fast (Circuit Breaker)
Behaviour	Keeps trying, hoping for eventual success	Stops trying after detecting a pattern of failures
Favours	Eventual success	Conserving resources
Cost	Extra wait time and work	Gives up on things that might have worked
Right for	Transient faults	Long-lasting or permanent faults

For transient faults, retries are ideal — there's a high chance the next attempt succeeds. For a service that will be down for hours, **blindly retrying wastes time and resources**, and circuit breakers or **graceful degradation** (serving defaults or **cached** data — saved copies of earlier answers) are more appropriate.

Use retries for hiccups; fall back or fail fast for lasting errors. They're complements, not competitors.

Part 4 — Architecture

The pattern is implemented as a **wrapper around an operation** that intercepts failures and repeats according to a policy.

Retry policy and parameters

Defines **when and how** to retry:

- The **maximum number of attempts**
- **Which exceptions or error codes** are considered retryable
- The **delay strategy** between attempts

Policies range from simple — *"retry up to 3 times with 2 seconds between tries"* — to sophisticated — *"retry 5 times with exponentially increasing delays, add randomness, and give up on certain error types."*

Retry loop (the invoker)

The mechanism that executes the operation and applies the policy. It calls the operation, catches any failure, checks whether a retry should happen per the policy, waits the specified delay, then calls again. This continues until either the operation succeeds or max retries is reached. If it never succeeds, the invoker **propagates the final failure** (passes the error up to its own caller), often after logging it as a persistent error.

Backoff strategy

To avoid hammering the failing resource, incorporate a wait between attempts rather than retrying in a **tight loop** (a loop that repeats as fast as the CPU allows, with no pause).

Strategy	Behaviour	When
Constant delay	Always wait the same, e.g. 5s	Simple cases; predictable recovery time
Exponential backoff	Each wait longer than the last: 1s, 2s, 4s	The general default
Capped exponential backoff	As above with a maximum	Prevents absurdly long waits

Exponential backoff **mitigates load on struggling services** by spacing attempts out, and increases the likelihood that by the time the next attempt runs, the transient issue has resolved.

The cap matters: without it, five doublings from one second gives you a 32-second wait, and eight gives you over four minutes. Implementations typically cap the maximum delay.

Jitter (randomization)

If many clients or threads experience an error at around the same time, **they might all retry in unison after the same backoff interval**. This produces the thundering herd — a burst of synchronized retries hitting the service simultaneously, potentially causing **a second failure spike**.

Jitter introduces **random variation in the delay**. Instead of every client waiting exactly 4 seconds, each waits a random time around 4 seconds — somewhere between 2 and 4, or with a random offset added. This **spreads out the bursts** so load on the downstream service stays more even.

There are several implementations — **full jitter**, **equal jitter**, **decorrelated jitter** — but the core idea is identical: introduce randomness so retries are **de-synchronized across clients**.

> It seems counter-intuitive to add randomness to a system deliberately, but it has been shown to significantly reduce **contention** (many clients competing for the same resource at once) and improve recovery times. This is one of those cases where the unintuitive answer is simply correct.

Circuit breaker integration

Retries deal with transient failures; circuit breakers handle persistent ones by cutting off calls to an unresponsive service after a threshold. In practice a breaker **prevents a retry loop from endlessly hitting a downed service**.

For example, if a service has been failing for the last 50 calls, the breaker opens and further calls fail immediately without even trying — giving the service time to recover. After a **cooldown** (a fixed waiting period), a **half-open** state (the breaker's trial mode) allows a test call, and success closes the circuit again.

The balance: allow 3 quick retries on a request, but if multiple requests in a row all fail — signalling a likely bigger problem — open the circuit to stop further traffic for a short period.

Idempotency and side effects

A critical design consideration. If an operation has **side effects** (changes it makes in the world beyond returning a value) — **charging a credit card, sending an email, incrementing a counter** — retrying could inadvertently perform the action **multiple times** unless you design against it.

Ideally, retried operations should be **idempotent**. Many **APIs** (interfaces that programs call) already are: deleting a resource, or performing a `GET` (the HTTP method for reading data). For non-idempotent actions — creating an order, making a payment — you must take special care.

The technique: use **unique transaction identifiers or tokens** (**idempotency keys** — a unique ID the client attaches to an operation and reuses on every retry of it) so the server can recognise a duplicate attempt and not process the action twice.

> The failure mode to picture: your payment call succeeds on the server, but the response is lost on the way back. Your client sees a timeout — indistinguishable from "it never happened" — and retries. **Without an idempotency key, the customer is charged twice.** The client genuinely cannot tell the difference between "lost request" and "lost response," which is why the server must be the one to **deduplicate** (detect and ignore repeats).

Placement in the architecture

Where should retries happen?

Client-side of a service-to-service call — the caller handles retrying. Service A calls Service B; A's client library or **HTTP client** (code that sends web requests) implements the retry policy. B remains **stateless** (keeps no memory between requests) and unaware of retries; it just sees repeated incoming requests.

API gateway or service mesh (a single entry point that routes outside requests, or a layer of proxies beside every service that routes internal traffic) — infrastructure automatically retries failed requests between services.

⚠️ **The key rule: avoid duplicating retries at multiple layers.**

> If every hop in a chain of microservices did its own retries, **a single failure could multiply into a storm of requests.** Three layers each retrying three times means one original request becomes twenty-seven. Good practice is to **centralize or delegate retries to one layer** — only at

the edge service, or only at the immediate caller — to prevent cascading retry explosions. In practice this means disabling internal retries in lower-level clients and handling it once at a higher layer, or using a coordinated approach like a **token bucket** to limit overall retry rates (a bucket refills with tokens at a fixed rate and each retry must spend one, so retries can never exceed that rate).

Part 5 — Worked example (Java)

```java
public class RetryUtil {

    public static <T> T executeWithRetry(Callable<T> task, int maxAttempts,
            long baseDelayMillis, double backoffFactor, boolean useJitter) throws Exception {

        int attempt = 1;
        Exception lastException = null;
        Random random = new Random();

        while (attempt <= maxAttempts) {
            try {
                return task.call();                 // success — return immediately
            } catch (Exception e) {
                lastException = e;                  // remember it for the final throw

                if (attempt == maxAttempts) {
                    break;                          // no retries left
                }

                // Work out how long to wait
                long delay = baseDelayMillis;
                if (backoffFactor > 1.0) {
                    // exponential: base * factor^(attempt-1)
                    delay = (long) (baseDelayMillis * Math.pow(backoffFactor, attempt - 1));
                }
                if (useJitter) {
                    // "full jitter": random value between 0 and the computed delay
                    delay = (long) (random.nextDouble() * delay);
                }
                // (a maximum cap could be applied here)

                try {
                    Thread.sleep(delay);
                } catch (InterruptedException ie) {
                    Thread.currentThread().interrupt();   // restore the interrupt flag
                    break;                                // stop retrying
                }

                attempt++;
            }
        }
        throw lastException;    // all attempts exhausted
    }
}
```

The parameters

Parameter	Meaning
`maxAttempts`	Total tries **including the initial attempt**. `3` means 1 try + 2 retries
`baseDelayMillis`	Fixed delay if constant; the initial delay if exponential
`backoffFactor`	The multiplier. `2.0` with base 100ms gives 100ms, 200ms, 400ms. Set `1.0` for a constant interval
`useJitter`	Applies full jitter — a random value between 0 and the computed delay

Reading the logic

`Callable<T>` makes it reusable. `Callable` is Java's interface for "an operation that returns a value and may throw." Accepting one means this utility wraps *any* operation — a network call, a database query — rather than being tied to one.

Success returns immediately. `return task.call()` exits the loop on the first success. No further attempts, no delay.

The exception is saved, not swallowed. `lastException = e` keeps the most recent failure so it can be thrown to the caller at the end. The caller learns *why* it ultimately failed.

The attempt check comes before the delay. If this was the last attempt, `break` immediately — there's no point sleeping before giving up.

`Math.pow(backoffFactor, attempt - 1)` — on attempt 1 that's `factor^0 = 1`, so you wait `baseDelay`. On attempt 2 it's `factor^1`, so double. And so on.

The `InterruptedException` handling is correct and worth copying. When a thread is **interrupted** (another thread politely asks it to stop what it's doing), catching the exception clears the interrupt flag. `Thread.currentThread().interrupt()` restores it so code higher up still knows the thread was asked to stop. Forgetting this is a classic Java bug that makes applications refuse to shut down.

Using it

```
String result = RetryUtil.executeWithRetry(
    () -> fetchData(url),
    3,        // up to 3 attempts
    1000,     // 1 second base delay
    2.0,      // exponential backoff, doubling
    true      // use jitter
);
```

This attempts the fetch and, on failure, retries up to 2 more times — waiting about 1s before the first retry and about 2s before the second, with randomness in those waits. If all attempts fail it throws after roughly 3 seconds of total waiting.

In many cases that one retry is all that's needed to ride out a transient glitch and succeed on the second try.

Design choices, and one to change

The course is explicit about these:

- **Generic `Callable<T>`** keeps the logic reusable across any operation type
- **Configurable backoff and jitter** allows switching strategies: constant with no jitter (`1.0`, `false`), exponential without jitter (`2.0`, `false`), or exponential with jitter (`2.0`, `true`). This flexibility is crucial for tuning to different scenarios

- `maxAttempts` **prevents infinite loops** — vital, since an unbounded retry loop could endlessly hammer a failing service

And one deliberate simplification you should not carry into production:

- **It catches broad** `Exception`, treating *any* failure as retryable. In a real scenario you should catch **only specific transient exceptions** — `IOException` (Java's input/output and network error), timeouts — and let others bubble up without retry. **Your error-filtering logic belongs here**, and without it you'll cheerfully retry validation errors three times.

A refinement on jitter: full jitter (`random × delay`) can produce a near-zero wait, which partially defeats the purpose of backing off. Many production implementations randomise between **50% and 100%** of the delay instead, keeping a meaningful minimum gap.

Worth adding: logging each retry attempt, **metrics** (numeric measurements such as a retry counter), and a circuit-breaker check that aborts retrying if a higher-level circuit is already open.

Part 6 — Performance implications and risks

Increased load on dependencies

Every retry is an **additional request**. When failures are genuinely rare this extra load is negligible and well worth the improved reliability.

But if a downstream service is failing *because it's overloaded*, retries make the problem worse. If Service A is hammering a slow Service B and starts retrying aggressively, B gets even more requests. In the worst case, many clients retrying a struggling service create a **retry storm that amplifies load exponentially**, potentially overwhelming it completely.

This is precisely why **backoff is crucial** (breathing room between attempts) and **limited attempts** matter — and why circuit breakers or throttling (deliberately limiting request rates) should be layered on for protection.

Increased latency for end users

Retrying lengthens overall response time. A single failed attempt might add a few seconds of delay.

Sometimes that's acceptable — **a slightly slower successful response beats an immediate error**. But overdone it degrades experience, and chained retries across a call chain accumulate.

The countermeasure: **set an overall timeout for the operation** alongside retries. For example decide that an API call should either succeed **within 5 seconds including all retries**, or not at all, to maintain a responsive **SLA** (service-level agreement — a promised level of speed or uptime). Without this, retry policies silently blow past your **latency budget** (the maximum time a request is allowed to take; **latency** is the time a request takes from start to finish).

Wasted work and resource consumption

Every retry that ultimately fails was wasted effort — CPU, **bandwidth** (network capacity), memory that produced no result. If the underlying issue isn't transient, **all retries fail and all that work is for nothing**, plus it strains the system.

Hence: detect non-transient failures quickly. If a response says "permission denied" or "invalid request," a retry won't help. Your retry logic should inspect error codes and keep a **blacklist of non-retryable errors** (a list of errors that must never be retried).

A subtler point worth remembering: **if the first attempt already took a long time** — nearly hitting the client timeout — starting another attempt that will likely also time out is counterproductive. Better to propagate an error than tie up threads on hopeless retries.

Cascading effects in complex systems

Service A calls B, B calls C. If **both** A and B have their own retry policies, a failure in C causes B to issue multiple attempts, **and** A to issue multiple attempts of its call to B. This multiplies load on both B and C.

Avoid uncoordinated retries at every layer. Decide **which layer retries** and let others fail fast. A common practice: the edge or client-facing service handles retries, while intermediate services propagate errors. Monitoring should detect when excessive retries occur, since it often indicates a downstream **incident** (a production problem needing attention).

Retry storms and thundering herd

If a popular service glitches, many instances of many services may all start retrying at once. Backoff and jitter avoid synchronisation — **jitter specifically tackles the thundering herd** by de-synchronising retry timing.

Other mitigations: **retry budgets** (allowing a certain number of retries per second), and **adaptive algorithms** that lengthen backoff while errors continue, avoiding constant pressure.

Idempotency revisited

Beyond correctness, duplicate side effects have a performance cost too — sending the same email twice uses twice the email-sending resource. **Do not retry non-idempotent operations blindly** unless you have deduplication keys or **check-pointing** (recording progress so a repeat can skip steps already done) to guard against repeated side effects.

The mitigation checklist

- Implement **exponential backoff** — don't hammer rapidly
- Add **jitter** — avoid synchronised retries
- **Limit attempts**, and limit total time spent retrying
- Use **circuit breakers** or fail-fast switches where failure is likely persistent
- **Distinguish error types** — retry only transient ones
- **Monitor and tune** — frequent retries signal a struggling dependency or a bad policy
- **Ensure idempotency**, or implement server-side request deduplication for critical actions
- **Test under failure conditions** — simulate a dependency going down or exhibiting high latency, and verify your retry logic actually improves resilience rather than overwhelming the system

---

Part 7 — Use cases

Web services

The internet is unreliable, and transient faults are constant. When interacting with a remote web service and hitting a transient fault, retrying is usually beneficial because the **cost asymmetry** favours it: the cost of an unsuccessful request is high (an error message to the user, or an expensive fallback process), while the cost of a retry is typically just the time and resources to send another request.

Database operations

Many database operations are inherently uncertain — network issues, concurrency conflicts, temporary unavailability.

The clearest example: you try to save a record while **another user is updating the same record**. Rather than throwing an error, retry after a short delay. This allows the conflicting **transaction** (a

group of database changes applied all-or-nothing) to complete and **paves the way for your operation to succeed**.

Microservice architectures

When one service depends on another, the failure of the latter can bring down the former. Retrying gives the failing service **a chance to recover** and makes your service more resilient — ensuring a temporary hiccup doesn't become a full-blown outage.

Part 8 — Real-world examples

Cloud service SDKs (AWS, Google Cloud). (An **SDK**, software development kit, is the official library for calling a platform from your code; **AWS** is Amazon Web Services.) Cloud providers build retry strategies into their client libraries. AWS SDKs automatically retry failed requests — a transient 5xx from a service, or a throttling error — using **exponential backoff and jitter**. This matters because network calls to cloud APIs occasionally time out or hit rate limits. Google Cloud's libraries have built-in retries for idempotent requests; a storage upload that times out is retried behind the scenes. These SDKs are battle-tested to balance retries and backoff, so **developers benefit from the pattern without even realising it**.

Netflix. As part of their overall resilience engineering, Netflix's internal libraries retry calls to microservices or **data stores** (databases, caches and the like) if a request fails or a **host** (the machine a service runs on) is unresponsive. If one instance fails to respond, the client quickly **retries on another instance** — perhaps after a brief backoff — instead of erroring out immediately. Combined with circuit breakers, this ensures sporadic failures don't disrupt the viewing experience.

Payment processing (Stripe, PayPal). Financial APIs frequently encounter transient failures communicating with banking networks or third-party systems. Rather than immediately failing a transaction and upsetting a customer, payment systems retry behind the scenes.

Critically, they **encourage the use of idempotency keys** on API requests. If a network issue occurs, the client can safely retry with the same key, and the server **knows not to duplicate the charge**. This design **explicitly embraces the Retry Pattern — it assumes clients will retry on failure.** The result: a temporary glitch between **merchant** (the shop) and **gateway** (the payment provider's service) doesn't lose the order; the merchant's system retries automatically and payment goes through **without double-charging**.

Message queues and event processing. In systems built on **RabbitMQ** or **Kafka** (popular message brokers — software that stores messages from producers until consumers read them), retry appears as **requeueing messages** (putting a message back to be read again) or using **dead-letter queues** (a separate queue where messages that keep failing are parked for inspection). If a **consumer** (the program reading messages) fails to process a message due to a temporary error — say it can't reach a database — it doesn't **acknowledge** the message (tell the broker it was handled), so the message returns to the queue to be tried again after a delay. This is the Retry Pattern in an **asynchronous** context (where the sender doesn't wait for the work to be done).

A real example: in order processing, if an attempt to reserve inventory fails due to a **lock timeout** (it waited too long for another transaction to release a database lock — a guard that lets only one writer touch a row at a time), the message can be retried a minute later, by which time the lock may be free. Many large e-commerce platforms rely on this in their **event-driven architectures** (systems where services react to messages about things that happened) to ensure **eventually consistent** outcomes (all copies of the data agree once processing catches up).

Part 9 — System design examples

E-commerce checkout. The user has filled their cart and clicks Buy. The system must charge their credit card. But the **payment gateway** (the third-party service that actually charges cards) **is temporarily unavailable**. Do you tell the user their purchase failed? Or retry, hoping the gateway returns shortly?

With the Retry Pattern you choose the latter, handling the temporary glitch gracefully. *(And this is precisely where idempotency keys are non-negotiable — the double-charge scenario above.)*

IoT device data collection. A system collects data from **IoT** (Internet of Things — sensors and gadgets connected to the internet) devices and stores it in a database. Devices send data periodically. But the **database is temporarily unavailable** due to a network glitch or scheduled maintenance. Should the system discard the data?

Retry provides a resilient solution: handle the temporary issue and **ensure no data is lost**.

Part 10 — When NOT to use retry

- The error is clearly **non-transient** — a configuration error or fatal exception. Retrying just delays the inevitable
- The downstream service is **known to be down for an extended period** (a planned outage). A retry loop only burns resources; a circuit breaker or fallback response is preferable
- **Strict latency requirements** — **real-time** systems (ones that must respond within a guaranteed, very short time) may prefer to fail fast rather than retry and violate their latency SLA
- The operation is **extremely expensive**, or has **side effects that can't be repeated** safely

In such cases, prefer alternatives: **manual compensation** (a person or process undoing the effects), **eventual reconciliation** (comparing records later and fixing mismatches) for asynchronous processes, or simply **alerting a human**.

> **Use retries where they make sense — transient, recoverable errors — and avoid them where they don't: permanent failures, or scenarios where retries could cause harm.**

Part 11 — Common beginner misconceptions

"More retries means more reliability." Only up to a point. Beyond a few attempts you're mostly adding latency and load. If three attempts with backoff didn't work, the fourth probably won't either.

"Retry and circuit breaker are alternatives." They're complements doing opposite things. Retry handles the blip; the breaker handles the outage. Use both.

"Jitter is a micro-optimisation." It's the difference between a graceful recovery and a self-inflicted outage at scale. At ten thousand clients it's essential, not decorative.

"My operation is a GET, so idempotency doesn't matter." True for that call — but check the whole path. A GET that triggers a server-side counter increment or **audit write** (recording who accessed what) is not as idempotent as it looks.

"Retries are free if they usually succeed." They're free when failures are rare. They're most expensive exactly when the system is already struggling — which is when they fire most.

"I'll retry at every layer to be safe." This is the most damaging mistake in the chapter. Layered retries multiply, turning one failure into an exponential request storm.

Part 12 — Check your understanding

1. Why is retrying an HTTP 401 pointless?
2. Explain the thundering herd, and how jitter fixes it.
3. Why can't a client distinguish "lost request" from "lost response," and what follows from that?
4. Why must you set an overall operation timeout alongside a retry policy?
5. If A retries B and B retries C, how many requests can one failure become at 3 attempts each?
6. What's the one line in the example code you should change before production use?

(Answers: 1 — authentication failure is permanent for that request; no number of attempts fixes wrong credentials; 2 — many clients fail simultaneously, back off by the same amount, and retry in unison, re-overloading the service; jitter randomises delays so retries spread out; 3 — both look like a timeout, so the server must deduplicate using an idempotency key rather than the client guessing; 4 — otherwise cumulative retry delays silently exceed your latency budget; 5 — nine, since each of A's 3 attempts triggers 3 of B's; 6 — catching broad `Exception`, which retries permanent errors too.)

Part 13 — Interview soundbite (30 seconds)

> "Retry handles transient failures — network blips, timeouts, throttling, a connection refused during a spike — things that fix themselves in seconds. You wrap the call in a policy: max attempts, which errors are retryable, and a delay strategy. Two details matter most. First, exponential backoff, so you're not hammering a service that may be failing precisely because it's overloaded. Second, jitter — random variation in the delay — because otherwise every client that failed at the same moment retries at the same moment, and you get a thundering herd that causes a second outage. You also need idempotency: the client can't distinguish a lost request from a lost response, so retrying a payment without an idempotency key double-charges the customer. That's why Stripe's API is built around them — it explicitly assumes clients will retry. And you have to distinguish transient from permanent, because retrying a 400 or a 401 is pure waste. The biggest architectural mistake is retrying at every layer: if A retries B and B retries C, one failure becomes nine requests, so you pick one layer to retry and everything else fails fast. Retry pairs with circuit breakers — retry for hiccups, trip the breaker for real outages — and you set an overall operation timeout so the retries can't blow your latency budget."

Key takeaways: retry transient only · backoff plus jitter · limit attempts · make it idempotent · retry at one layer · test under failure.

Part 14 — Implementation

A. Runnable demo — backoff, jitter, and the thundering herd

Full source: `demos/chapters/08-retry/RetryDemo.java`

```
cd demos && ./gradlew runCh08
```

```java
import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicInteger;

public class RetryDemo {

    // ---------- Which errors are worth retrying ----------
    static class TransientException extends RuntimeException {
        TransientException(String m) { super(m); }
    }
    static class PermanentException extends RuntimeException {
        PermanentException(String m) { super(m); }
    }

    // ---------- The retry utility ----------
    static <T> T executeWithRetry(Callable<T> task, int maxAttempts,
                                  long baseDelayMs, double factor,
                                  boolean useJitter) throws Exception {
        Random random = new Random();
        Exception last = null;

        for (int attempt = 1; attempt <= maxAttempts; attempt++) {
            try {
                T result = task.call();
                if (attempt > 1) System.out.println("      succeeded on attempt " + attempt);
                return result;
            } catch (PermanentException e) {
                System.out.println("      permanent error, NOT retrying: " + e.getMessage());
                throw e;                                  // fail fast
            } catch (Exception e) {
                last = e;
                if (attempt == maxAttempts) break;

                long delay = (long) (baseDelayMs * Math.pow(factor, attempt - 1));
                delay = Math.min(delay, 5000);            // cap
                if (useJitter) {
                    // "equal jitter": 50-100% of the computed delay
                    delay = delay / 2 + (long) (random.nextDouble() * delay / 2);
                }
                System.out.printf("      attempt %d failed (%s), waiting %dms%n",
                        attempt, e.getMessage(), delay);
                Thread.sleep(delay);
            }
        }
        throw last;
    }

    // ---------- A service that fails N times then works ----------
    static class FlakyService {
        private final AtomicInteger calls = new AtomicInteger();
        private final int failuresBeforeSuccess;

        FlakyService(int n) { this.failuresBeforeSuccess = n; }

        String call() {
            int n = calls.incrementAndGet();
            if (n <= failuresBeforeSuccess) throw new TransientException("timeout");
            return "OK after " + n + " calls";
        }
    }

    public static void main(String[] args) throws Exception {
```

```
        System.out.println("\n=== 1. Transient failure, recovers on 3rd attempt ===");
        FlakyService flaky = new FlakyService(2);
        System.out.println("      result: " +
                executeWithRetry(flaky::call, 5, 100, 2.0, false));

        System.out.println("\n=== 2. Permanent failure — should NOT retry ===");
        try {
            executeWithRetry(() -> { throw new PermanentException("HTTP 401"); },
                    5, 100, 2.0, false);
        } catch (PermanentException e) {
            System.out.println("      gave up immediately (correct)");
        }

        System.out.println("\n=== 3. Backoff schedule, no jitter ===");
        try {
            executeWithRetry(() -> { throw new TransientException("503"); },
                    5, 100, 2.0, false);
        } catch (Exception ignored) { }

        System.out.println("\n=== 4. Same schedule WITH jitter ===");
        try {
            executeWithRetry(() -> { throw new TransientException("503"); },
                    5, 100, 2.0, true);
        } catch (Exception ignored) { }

        System.out.println("\n=== 5. Thundering herd: 1000 clients retry at once ===");
        demonstrateHerd(false);
        demonstrateHerd(true);
    }

    // Show how many clients land in the same 100ms bucket
    static void demonstrateHerd(boolean useJitter) {
        Random random = new Random();
        Map<Long, Integer> buckets = new TreeMap<>();
        long baseDelay = 1000;

        for (int i = 0; i < 1000; i++) {
            long delay = baseDelay;
            if (useJitter) {
                delay = delay / 2 + (long) (random.nextDouble() * delay / 2);
            }
            buckets.merge(delay / 100 * 100, 1, Integer::sum);
        }

        int worst = buckets.values().stream().mapToInt(Integer::intValue).max().orElse(0);
        System.out.printf("      jitter=%-5s -> busiest 100ms bucket holds %d of 1000 clients%n",
                useJitter, worst);
    }
}
```

B. What you should see

```
=== 1. Transient failure, recovers on 3rd attempt ===
      attempt 1 failed (timeout), waiting 100ms
      attempt 2 failed (timeout), waiting 200ms
      succeeded on attempt 3
      result: OK after 3 calls

=== 2. Permanent failure — should NOT retry ===
      permanent error, NOT retrying: HTTP 401
      gave up immediately (correct)

=== 3. Backoff schedule, no jitter ===
      attempt 1 failed (503), waiting 100ms
      attempt 2 failed (503), waiting 200ms
      attempt 3 failed (503), waiting 400ms
      attempt 4 failed (503), waiting 800ms      <- doubling

=== 4. Same schedule WITH jitter ===
      attempt 1 failed (503), waiting 74ms
      attempt 2 failed (503), waiting 133ms
      attempt 3 failed (503), waiting 291ms      <- randomised
      attempt 4 failed (503), waiting 512ms

=== 5. Thundering herd: 1000 clients retry at once ===
      jitter=false -> busiest 100ms bucket holds 1000 of 1000 clients
      jitter=true  -> busiest 100ms bucket holds ~205 of 1000 clients
```

Section 5 is the one to internalise. Without jitter, **all 1000 clients retry in the same 100ms window** — a synchronised wall of traffic hitting a service that just recovered. With jitter the worst bucket holds about a fifth of that. Same number of retries, spread over time instead of stacked.

C. Production — Resilience4j Retry

(**Resilience4j** is the standard Java library for resilience patterns; this config is YAML, an indentation-based settings format.)

```yaml
resilience4j:
  retry:
    instances:
      inventoryService:
        maxAttempts: 3                   # 1 initial + 2 retries
        waitDuration: 200ms
        enableExponentialBackoff: true
        exponentialBackoffMultiplier: 2
        enableRandomizedWait: true       # <- jitter
        randomizedWaitFactor: 0.5
        retryExceptions:
          - java.io.IOException
          - java.util.concurrent.TimeoutException
          - org.springframework.web.client.HttpServerErrorException   # 5xx
        ignoreExceptions:
          - org.springframework.web.client.HttpClientErrorException   # 4xx — never retry
```

```java
@Retry(name = "inventoryService", fallbackMethod = "inventoryFallback")
@CircuitBreaker(name = "inventoryService")     // order matters — see pitfall 6
public Inventory check(String sku) {
    return inventoryClient.get(sku);
}
```

Spring's own `@Retryable` (an **annotation** — a label on a method — from Spring, the main Java web framework) is an alternative if you're not using Resilience4j:

```
@Retryable(
    retryFor = { IOException.class },
    noRetryFor = { ValidationException.class },
    maxAttempts = 3,
    backoff = @Backoff(delay = 200, multiplier = 2, random = true))
public Inventory check(String sku) { ... }

@Recover
public Inventory recover(IOException e, String sku) { return Inventory.unknown(sku); }
```

D. Implementing idempotency keys

The most important production concern, and the part usually missing from tutorials. The client sends a stable key (usually a **UUID** — a randomly generated, practically unique 128-bit ID — sent in an HTTP **header**, a key-value field alongside the request); the server guarantees one execution per key.

Client:

```
String idempotencyKey = UUID.randomUUID().toString();   // generated ONCE, before the first attempt

executeWithRetry(() ->
    http.post("/payments")
        .header("Idempotency-Key", idempotencyKey)      // same key on EVERY retry
        .body(payment)
        .execute(),
    3, 200, 2.0, true);
```

> **The critical detail:** generate the key **outside** the retry loop. A key generated inside means each attempt looks like a brand-new request and the customer gets charged three times — which is exactly the bug the mechanism exists to prevent.

Server:

```
@PostMapping("/payments")
public ResponseEntity<PaymentResult> pay(
        @RequestHeader("Idempotency-Key") String key,
        @RequestBody Payment payment) {

    // Atomic "insert if absent" — the DB unique constraint does the work
    Optional<PaymentResult> existing = idempotencyStore.find(key);
    if (existing.isPresent()) {
        return ResponseEntity.ok(existing.get());     // replay the original response
    }

    PaymentResult result = paymentService.charge(payment);
    idempotencyStore.save(key, result, Duration.ofHours(24));
    return ResponseEntity.ok(result);
}
```

Store keys in **Redis** (a fast in-memory key-value database) with a **TTL** (time-to-live — an expiry after which the entry is deleted), or a table with a **unique index** on the key (a database rule that rejects a second row with the same value). The **unique constraint** is what makes it safe under concurrent duplicate requests.

E. Implementation pitfalls

1. Retrying non-idempotent operations without a key. The headline risk: double charges, duplicate orders, duplicate emails.

2. Generating the idempotency key inside the retry loop. See above — silently defeats the whole mechanism.

3. Retrying 4xx errors. A 400 or 401 will fail identically every time. Always configure `ignoreExceptions`.

4. No cap on exponential backoff. Base 1s with multiplier 2 and 10 attempts ends at a **17-minute** wait. Always set a maximum.

5. No overall deadline (one time limit for the whole operation, all attempts included). Individual timeouts plus retries multiply: a 5s timeout with 3 attempts plus backoff is potentially 20 seconds. Set a budget for the whole operation:

```
resilience4j.timelimiter.instances.inventoryService.timeoutDuration: 3s
```

6. Wrong ordering of retry and circuit breaker. Resilience4j's default order is **Retry(CircuitBreaker(call))** — the breaker is *inside*, so retries happen around it and the breaker sees each attempt. That's usually what you want. If you invert it, the breaker counts a whole exhausted retry sequence as a single failure and opens far too slowly.

7. Retrying at every layer. If the HTTP client retries 3×, the service retries 3×, and the gateway retries 3×, one user request becomes **27** requests to a service that's already struggling. Pick one layer. Disable retries everywhere else explicitly — most HTTP clients have them on by default.

8. Retrying immediately with no delay. A tight retry loop against a struggling service is indistinguishable from an attack.

9. Not logging retries. If retries are invisible, a dependency degrading from 0.1% to 15% failure looks totally healthy from the outside — until it tips over. Emit a metric per retry.

Related patterns

- **Circuit Breaker** — the essential companion; stops retries becoming an attack on a downed service
- **Bulkhead** — bounds the resources retries can consume
- **Service Discovery** — retry against a *different* discovered instance
- **Saga** — compensation for when retrying isn't safe or sufficient
- **Event-Driven Architecture** — requeueing and dead-letter queues are retry in an async context